

Course Project

May 2026

Guidelines

- The project is due on 23:59, June 7th.
- Full score is **100 points**. Partial credit will be given for partially correct implementation and clear analysis.
- Collaboration, online search, and the use of LLM-based agents are allowed. You **must explicitly acknowledge** all collaborators, sources, and AI tools used in your report.
- Submit a single .zip file (no larger than 20 MB) containing your report as a .pdf and a folder with all code.

This project has two main choices. You only need to complete **one** of them. We will select a small number of students with the best work to give a presentation of about 10 minutes in class, and award them a **bonus of up to 10 pts in this project** (which translates to roughly 2 pts in the final course grade).

Part	Main Focus	Score
Choice A	Part 0: Environment setup	
	Part 1: Implement SFT, run training, and report evaluation	20 pts
	Part 2: Implement RL, run training, and report evaluation	40 pts
	Part 3: Open exploration	40 pts
Choice B	Reproduce one paper and finish a report	100 pts

Choice A: Post-training Implementation

Modern LLMs are typically built in two stages: pre-training on large-scale corpora, followed by post-training — including SFT and RL — to improve instruction-following and align the model with human preferences. In this path, you will implement the post-training pipeline, run experiments, and compare model behaviors before and after post-training.

Part 0: Environment Setup

Before you begin, create a clean environment. In practice, version mismatch is one of the most common reasons why LLM training code fails. For this project, the following environment is recommended:

```
conda create -n llm_project python=3.10
conda activate llm_project

pip install vllm==0.9.0 transformers==4.51.1 datasets torchdata deepspeed
wget https://github.com/Dao-AILab/flash-attention/releases/download/v2.8.2/
flash_attn-2.8.2+cu12torch2.7cxx11abiFALSE-cp310-cp310-linux_x86_64.whl
```

```
pip install flash_attn-2.8.2+cu12torch2.7cxx11abiFALSE-cp310-cp310-  
linux_x86_64.whl
```

Important

- Compiling flash-attn from source is extremely slow (around 1 hour), so we recommend downloading a pre-built wheel directly from its [GitHub release page](#), as shown in the script above.
- Most version adjustments (python, vllm, other dependencies) are fine, but the flash-attn wheel must **strictly match** your environment's CUDA version, PyTorch version, Python version, and CXX11 ABI setting — a mismatch in any of these will cause failures.

Part 1: SFT

Supervised Fine-Tuning (**SFT**) is the standard first step in post-training. The basic idea is simple: given an input and a high-quality target response, we train the language model to imitate that response with the next-token prediction objective. In modern LLM pipelines, SFT is often the step that makes a base model usable for downstream tasks.

You will implement an SFT training pipeline from scratch, finetune [Qwen2.5-1.5B-Base](#) on the subset of the [NuminaMath-CoT](#) training set with source=gsm8k (7345 samples), and evaluate on the test set of both [GSM8K](#) and [MMLU](#).

(1) Implement SFT. Build your SFT pipeline from scratch. Your implementation must support multi-GPU training, and using [DeepSpeed](#)¹ is recommended.

(2) Run SFT. Train your SFT model and record the training / validation curve.

(3) Evaluate your model. Evaluate the model *before SFT* and *after SFT* on both GSM8K and MMLU.

What To Submit

- Your full SFT codebase.
- One figure containing the **train / validation curve**.
- One comparison table reporting **before-SFT** and **after-SFT** results on GSM8K and MMLU.
- A summary of your observations.

Important

- Grading is based on both the experiment report and the correctness of your implementation.

Part 2: RL

Reinforcement learning (RL) enters the picture when imitation alone is no longer enough. Instead of asking the model to imitate a fixed answer token by token, we ask it to *sample* responses,

¹<https://github.com/microsoft/DeepSpeed>

receive a scalar reward, and gradually shift probability mass toward better responses. This is particularly useful for reasoning problems, where multiple candidate responses may be plausible but clearly different in quality.

In this section you will implement a Group Relative Policy Optimization (GRPO)² training pipeline from scratch and apply it to `Qwen2.5-1.5B-Base` on GSM8K.

(1) Implement GRPO. Build your GRPO pipeline from scratch. Your implementation must support multi-GPU training, and using **DeepSpeed** is recommended. Using `vLLM` to accelerate rollout generation is optional but not required at this stage.

(2) Run GRPO. Train your model on the training set of GSM8K and record the reward curve and test-set accuracy curve during training.

(3) Evaluate your model. Evaluate the model *before GRPO* and *after GRPO* on both GSM8K and MMLU.

What To Submit

- Your full GRPO codebase.
- One figure containing the **reward curve** and **test-set accuracy curve**.
- One comparison table reporting **before-GRPO** and **after-GRPO** results on GSM8K and MMLU.
- A summary of your observations.

Important

- Grading is based on both the experiment report and the correctness of your implementation.

Part 3: Open Exploration

In this final part, you are encouraged to go one step further and try something of your own. The goal is not necessarily to obtain a great improvement, but to identify a meaningful direction, make a concrete modification, run a controlled experiment, and explain what you observe.

You do **not** need to do everything here. One well-executed direction is much better than several incomplete attempts.

- 1) **Choose one extension direction.** A few possible options are listed below.
- 2) **Implement or test it.** Make the change in a concrete and reproducible way.
- 3) **Report what happens.** Summarize the setup, the result, and your interpretation.

Possible Directions

- **Support more model or data settings.** Try to support MoE models, or extend the current pipeline to multi-turn settings.
- **Accelerate rollouts with vLLM.** Integrate `vLLM` into the rollout generation step of your RL training, and report the speedup compared to a non-`vLLM` baseline.

²<https://arxiv.org/abs/2402.03300>

- **Try some engineering work.** Try to implement sequence packing, sequence balancing, or other efficiency-oriented improvements that make training more stable or more efficient.
- **Try different RL algorithms.** Compare GRPO with other RL variants.

What To Submit

- A brief description of the direction you chose.
- Figures, tables, or comparison results that reflect the outcome of your exploration.
- Explanations of what worked, what did not, or other interesting observations.
- It is expected that you spend about 4–6 hours on this part, with roughly 2–4 pages in the report.

Choice B: Reproduce One Paper

If you choose this path, you do **not** need to implement the SFT / RL code above. Instead, you should reproduce one recent paper and write a report. The expectation is that you successfully reproduce the **main conclusions** of the paper.

Using a data subset is allowed, as long as your setup remains technically meaningful and the report clearly explains what was kept and what was simplified.

We provide two suggested papers:

- Distillation Robustifies Unlearning³
- From $f(x)$ and $g(x)$ to $f(g(x))$: LLMs Learn New Skills in RL by Composing Old Ones⁴

Other papers from this paper list⁵ are also acceptable. If there is another paper that you personally find especially interesting, you may use that as well, and briefly explain why you chose it.

Tasks

- 1) Choose one recent ML paper from the list above, or one of your own choosing.
- 2) Reproduce the main conclusions of the paper.
- 3) Write a report of 4–8 pages (excluding references and appendix), covering the following:
 - **Introduction:** briefly introduce the target paper, including its background, core idea, method, and main conclusions.
 - **Setup:** the model, dataset, and hyperparameters you used.
 - **Changes:** any simplifications or deviations from the original paper.
 - **Results:** what you were able to reproduce and to what extent.
 - **Discussion:** what you learned, what surprised you, and any limitations or future directions you see.

³<https://arxiv.org/abs/2506.06278>

⁴<https://arxiv.org/abs/2509.25123>

⁵https://cloudygoose.github.io/nlp_papers.html

Important

You should implement the core logic yourself rather than merely running the official codebase. Referencing the official code is acceptable, but your submitted implementation should reflect your own understanding and effort.

What To Submit

- The codebase used for reproduction.
- A report as described above.